Storage Architecture

For KVM & KubeVirt — From Local NVMe to Distributed Ceph

Choosing the right storage backend is the most impactful architectural decision after migration. This guide covers local NVMe, LVM thin-provisioning, Ceph/Rook, Longhorn, OpenEBS, NFS, and ZFS. Includes QCOW2 features, performance data, and a sizing calculator.

NVMe | LVM Thin | Ceph/Rook | Longhorn | QCOW2 | ZFS | Sizing Calculator

Storage Backend Comparison

Seven storage options for KVM and KubeVirt, ranked by performance and complexity.

Storage Decision Matrix

Backend	IOPS (4K)	Throughput	HA	Complexity	Cost	KVM	KubeVirt	Best For
Local NVMe	500K+	3.5 GB/s	No	Trivial	$$	Native	hostPath	Databases, highest perf
LVM Thin	400K+	3.0 GB/s	No	Low	$	Native	TopoLVM	Multi-VM local, snapshots
Ceph/Rook	80-120K	1.5 GB/s	Yes	High	$$$	RBD	Rook CSI	Enterprise HA, large scale
Longhorn	40-60K	800 MB/s	Yes	Medium	$$	No	CSI	K8s-native, simple HA
OpenEBS	50-80K	900 MB/s	Yes	Medium	$$	No	CSI	K8s-native, Mayastor
NFS	20-40K	1.0 GB/s	Depends	Low	$	Native	CSI	Shared, simple, legacy
ZFS	300K+	2.5 GB/s	No	Medium	$$	zvol	TopoLVM	Compression, checksums

Storage Architecture Patterns

Local NVMe (Direct Attach)

- Best raw performance: native NVMe latency (<100us)
- QCOW2 on XFS/ext4 filesystem, or raw LVM logical volumes
- No network overhead, no replication overhead
- VM tied to host (no live migration without shared storage)
- Ideal for: single-host, database VMs, edge deployments
- Sizing: 1 NVMe per 5-10 VMs (depends on workload)

LVM Thin Provisioning

- Thin pool: allocates space on write (overcommit safe)
- `lvcreate --thin -L 500G vg0/thinpool`
- `lvcreate -V 100G --thin vg0/thinpool -n vm-disk`
- Snapshots: `lvcreate -s -n snap vg0/vm-disk` (instant, CoW)
- Near-NVMe performance (minimal overhead vs raw)
- KubeVirt: TopoLVM CSI driver manages LVM volumes

Ceph / Rook (Distributed)

Longhorn (K8s-Native)

- 3-node minimum for replication factor 3
- RBD (RADOS Block Device) for VM disks
- KVM: `qemu-img create -f raw rbd:pool/image 100G`
- KubeVirt: Rook CSI provisioner auto-creates RBD volumes
- Live migration works: RBD is network-accessible from all nodes
- Self-healing: auto-rebalances on node failure

- Designed for Kubernetes, installed via Helm chart
- Replication: 2 or 3 replicas across nodes (configurable)
- Backup to S3/NFS with incremental snapshots
- UI dashboard for volume management
- iSCSI-based: each volume is an iSCSI target
- Good for: small-medium K3s/KubeVirt clusters (<20 nodes)

QCOW2 Features & Optimization

QCOW2 is the default disk format for KVM. Understanding its features is essential.

QCOW2 Feature Overview

Thin Provisioning (Copy-on-Write)

- QCOW2 allocates clusters (64KB default) only on first write
- A 100GB virtual disk may use only 15GB on host
- Overcommit: allocate more virtual than physical (with monitoring)
- TRIM/discard: `fstrim -av` in guest reclaims unused clusters
- Check actual usage: `qemu-img info --output=json disk.qcow2`
- Defragment: `qemu-img convert -O qcow2 old.qcow2 new.qcow2`

Snapshots

- Internal snapshots: stored within the QCOW2 file itself
- `qemu-img snapshot -c snap1 disk.qcow2`
- External snapshots: separate overlay file (better for production)
- `virsh snapshot-create-as vm snap1 --disk-only`
- Revert: `qemu-img snapshot -a snap1 disk.qcow2`
- Performance: CoW snapshots have minimal write overhead

Compression

- Cluster-level zlib compression: `qemu-img convert -c -O qcow2`
- Typical ratio: 40-60% size reduction for OS disks
- Read: decompressed on-the-fly (minor CPU overhead)
- Write: new writes go to uncompressed clusters
- Best for: archived VMs, templates, backups
- Not recommended for active database VMs (CPU overhead)

Encryption (LUKS)

- QCOW2 supports native LUKS encryption (AES-256-XTS)
- `qemu-img create -f qcow2 --object secret,id=sec -o encrypt.format=luks`
- Performance: ~10-15% overhead with AES-NI acceleration
- Key management: libvirt secret objects or external KMS
- Encryption at rest: protects disk image on host filesystem
- Compliance: satisfies HIPAA, PCI DSS encryption requirements

QCOW2 Tuning Parameters

```
# Create optimized QCOW2 for production
qemu-img create -f qcow2 \
    -o cluster_size=2M \           # 2MB clusters (better sequential I/O, default is 64KB)
    -o lazy_refcounts=on \         # Defer refcount updates (faster writes, fsck on crash)
    -o preallocation=metadata \    # Pre-allocate metadata (avoids fragmentation)
    -o extended_l2=on \            # Extended L2 entries (subcluster allocation, QEMU 5.2+)
    disk.qcow2 100G

# Convert existing disk with optimization
qemu-img convert -f vmdk -O qcow2 \
    -o cluster_size=2M,lazy_refcounts=on,preallocation=metadata \
```

```
    source.vmdk optimized.qcow2

# Check QCOW2 health
qemu-img check disk.qcow2
# Output: No errors found, image is clean
```

QCOW2 Cluster Size Impact

Cluster Size	Random 4K IOPS	Sequential MB/s	Metadata Size	Best For
64 KB (default)	**Best (no waste)**	Good	Smallest	General purpose, random I/O
256 KB	Good	Better	Small	Mixed workloads
2 MB	Slightly lower	**Best**	Larger	Sequential I/O, databases, backups

NFS Storage Configuration

NFS Server Setup

```
# /etc/exports (NFS server)
/srv/vms 192.168.1.0/24(rw,sync,
  no_subtree_check,no_root_squash)

# Enable NFS 4.2 for better performance
# Server: nfsconf --set nfsd vers4.2 y
# Mount: -o vers=4.2,proto=tcp
```

libvirt NFS Pool

```
# Define NFS storage pool
virsh pool-define-as nfs-pool \
  netfs --source-host nfs-server \
  --source-path /srv/vms \
  --target /var/lib/libvirt/nfs

virsh pool-start nfs-pool
virsh pool-autostart nfs-pool
```

ZFS & Advanced Storage

ZFS on Linux provides checksums, compression, and snapshots with near-native performance.

ZFS for KVM Virtual Machines

ZFS Pool Setup

```
# Create mirrored pool (2 NVMe)
zpool create vmpool mirror \
  /dev/nvme0n1 /dev/nvme1n1

# Create dataset for VM images
zfs create vmpool/vms
zfs set compression=lz4 vmpool/vms
zfs set atime=off vmpool/vms
zfs set recordsize=64k vmpool/vms

# Create zvol for raw VM disk
zfs create -V 100G vmpool/vms/vm1
```

ZFS Benefits for VMs

- **Checksums:** every block verified (no silent corruption)
- **Compression:** LZ4 transparent, ~1.5x ratio, near-zero CPU
- **Snapshots:** `zfs snapshot vmpool/vms/vm1@backup` (instant)
- **Send/Recv:** `zfs send -i @snap1 @snap2 | ssh remote zfs recv`
- **ARC cache:** adaptive replacement cache in RAM (self-tuning)
- **L2ARC:** SSD read cache for spinning disk pools

Ceph/Rook Deployment for KubeVirt

```
# Install Rook operator (Helm)
helm repo add rook-release https://charts.rook.io/release
helm install rook-ceph rook-release/rook-ceph -n rook-ceph --create-namespace

# Create CephCluster CR
apiVersion: ceph.rook.io/v1
kind: CephCluster
metadata:
  name: rook-ceph
  namespace: rook-ceph
spec:
  dataDirHostPath: /var/lib/rook
  mon:
    count: 3
  storage:
    useAllNodes: true
    useAllDevices: true
    deviceFilter: "^nvme"        # Use NVMe devices only

# Create RBD StorageClass for KubeVirt
```

```yaml
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: ceph-block
provisioner: rook-ceph.rbd.csi.ceph.com
parameters:
  pool: replicapool
  imageFormat: "2"
  imageFeatures: layering,exclusive-lock,object-map,fast-diff
reclaimPolicy: Delete
allowVolumeExpansion: true
```

OpenEBS Mayastor for High-Performance KubeVirt Storage

Mayastor Architecture

- User-space storage engine using SPDK (Storage Performance Development Kit)
- NVMe-oF (NVMe over Fabrics) for network transport
- Bypasses Linux kernel I/O stack entirely
- Synchronous replication across nodes
- CSI driver for automatic PV provisioning
- Performance: 80-90% of local NVMe IOPS

Longhorn for Simplicity

- Lightweight: installs via single Helm chart
- Built-in UI: volume management, snapshot, backup
- Backup to S3: incremental, scheduled, encrypted
- DR: cross-cluster volume replication
- Resource usage: ~200MB RAM per node
- Best for: K3s clusters, edge, small-medium deployments

VMware vSAN to KVM Storage Migration Mapping

VMware Storage	KVM Equivalent	KubeVirt Equivalent	Notes
VMFS datastore (local)	QCOW2 on XFS/ext4	hostPath or TopoLVM	Simplest migration path
vSAN (distributed)	Ceph RBD	Rook-Ceph CSI	Similar HA model, more flexible
NFS datastore	NFS mount + QCOW2	NFS CSI provisioner	Direct equivalent
vVols (SAN)	iSCSI/FC LUNs	SAN CSI driver	Vendor-specific CSI
VMware Thick Eager Zero	QCOW2 preallocation=full	Not applicable	Best performance, wastes space
VMware Thin Provision	QCOW2 (default thin)	PVC (thin by default)	Space-efficient, standard

Storage Sizing Calculator

Plan your storage capacity for migrated VMs with realistic ratios.

Capacity Planning Formula

Effective Capacity = Raw Capacity / (Replication Factor x Overhead Factor)
Example: 12 x 4TB NVMe = 48TB raw. With Ceph 3x replication + 25% overhead: 48 / (3 x 1.25) = **12.8 TB usable**.
With LVM thin (no replication) + 10% overhead: 48 / 1.10 = **43.6 TB usable**.

Sizing Examples by Deployment Size

Scale	VMs	Virtual Size	Actual Usage	Storage Backend	Raw Required	Hardware
Small / Edge	5-20	1-4 TB	500 GB-2 TB	Local NVMe + LVM	2x 2TB NVMe	1 server
Medium	20-100	4-20 TB	2-10 TB	ZFS mirror or Longhorn	3 servers x 4TB	3-node cluster
Large	100-500	20-100 TB	10-50 TB	Ceph/Rook (3x repl)	5+ servers x 20TB	5+ OSD nodes
Enterprise	500+	100+ TB	50+ TB	Ceph/Rook + SSD tier	10+ servers x 40TB	Dedicated storage tier

Thin Provisioning Ratios (Observed from 500+ Migrations)

Linux VMs

VM Type	Virtual Size	Actual Usage	Ratio
Web server (nginx)	50 GB	8 GB	6.2:1
App server (Java)	100 GB	25 GB	4:1
Database (PostgreSQL)	200 GB	140 GB	1.4:1
File server	500 GB	380 GB	1.3:1

Windows VMs

VM Type	Virtual Size	Actual Usage	Ratio
Windows Server (base)	80 GB	32 GB	2.5:1
SQL Server	200 GB	160 GB	1.25:1
Active Directory	60 GB	18 GB	3.3:1
Desktop (VDI)	100 GB	45 GB	2.2:1

QCOW2 Compression Savings (Post-Migration)

```
# Compress during migration (hyper2kvm --compress)
$ h2kvmctl convert --source vm.vmdk --format qcow2 --compress --output /var/lib/libvirt/images/

# Before compression: 42.8 GB on disk (100 GB virtual)
# After compression:  18.3 GB on disk (57% reduction)
# Read performance:   -3% (negligible with modern CPUs)
# Write performance:  No impact (new writes are uncompressed)

# Offline re-compress an existing QCOW2
$ qemu-img convert -c -O qcow2 vm.qcow2 vm-compressed.qcow2
```

Storage Recommendation by Use Case

Single server / Edge: Local NVMe + LVM thin (simplest, fastest).

3-node HA cluster: Longhorn or ZFS + replication (K8s-native, easy).

Enterprise / 100+ VMs: Ceph/Rook (proven at scale, self-healing).

hyper2kvm outputs QCOW2 by default — thin provisioned, snapshot-ready, compression-optional.